

Índice

1. Descripción del tutorial	2
2. Introducción a mShield.....	2
3. Parámetros de especificación	3
4. Especificaciones de la interfaz	4
5. Conceptos básicos del editor de Python	5
5.1 Crear un nuevo proyecto	6
5.2 Guardar el proyecto localmente	7
5.3 Importar un proyecto local	7
5.4 Subir el proyecto	8
5.5 Guardar el proyecto como archivo Python	13
5.6 Aprendizaje de la sintaxis básica del editor de Python	13
6. Código de ejemplo	14
6.1 Versión del firmware	15
6.2 LED	17
6.3 PWM	19
6.4 Nivel de batería	21
6.5 Motor	23
6.6 180 Servo de grados	27
6.7 360 Servo de grado	30
7. Control de calidad	34
7.1 No se puede cargar el código en el micro:bit	34
7.2 La letra de unidad de micro:bit se muestra incorrectamente	34
7.3 El motor sigue girando tras volver a cargar el código	34
7.4 El motor no funciona	34
7.5 Otros fallos	35
8. Contáctanos	35

1. Descripción del tutorial

1.1 Python es un lenguaje de programación de uso general que se ejecuta en hardware potente (como ordenadores de sobremesa y servidores). MicroPython es una versión simplificada de Python diseñada para microcontroladores y dispositivos integrados.

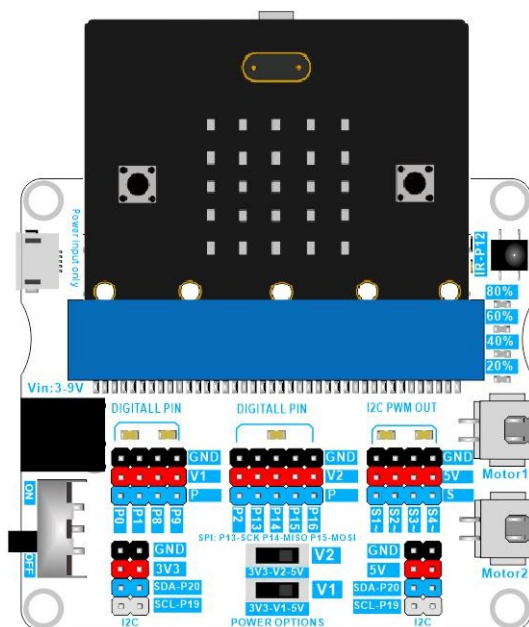
1.2 Este tutorial utiliza MicroPython para la programación. El micro:bit no dispone de una biblioteca dedicada de MicroPython para infrarrojos NEC. Si se utiliza un pin para leer señales de infrarrojos NEC, la velocidad de lectura es demasiado lenta, lo que provoca una pérdida de señal considerable. Por lo tanto, este tutorial no puede implementar un mando a distancia por infrarrojos NEC.

Copyright © Equipo SIYEENOVE. Todos los derechos reservados. El equipo SIYEENOVE se reserva el derecho a actualizar el contenido de este tutorial en cualquier momento.

2. Introducción a mShield

La placa de expansión mShield es nuestra última placa de expansión para micro:bit, muy fácil de usar. Integra potentes funciones, como control de motores de 2 canales, recepción de infrarrojos, lectura del nivel de batería, control de servos de 4 canales, salida de señal PWM de 4 canales, indicadores LED y salida de alimentación seleccionable de 3 V o 5 V.

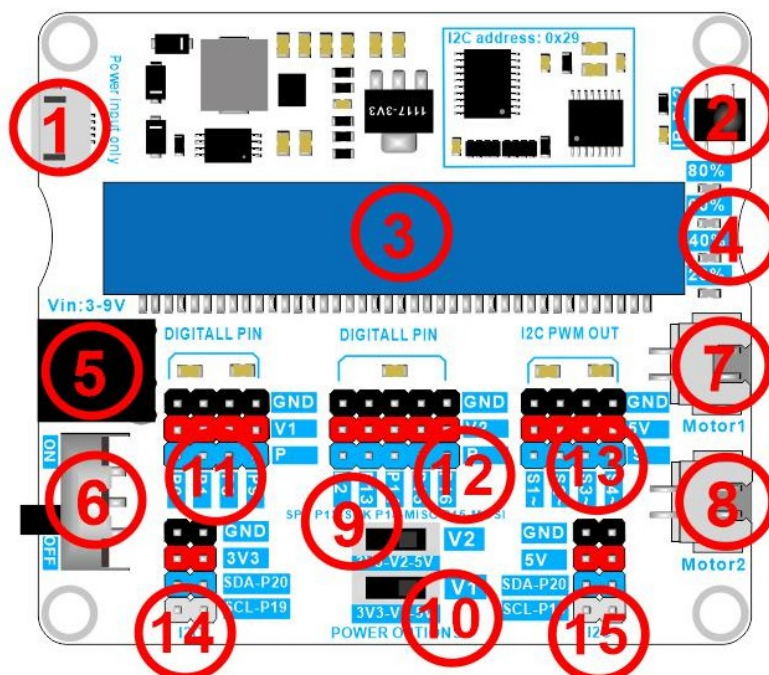
La placa de expansión amplía los pines más utilizados del micro:bit en forma de conectores de pines, lo que permite la conexión a otros sensores. La placa de expansión se puede conectar a dos tipos de baterías multicelulares y también cuenta con un puerto micro USB, lo que permite la conexión a un cargador portátil, ofreciendo una mayor flexibilidad en la alimentación y una mayor seguridad. Para facilitar la fijación de la placa de control a otros dispositivos, la placa de expansión incluye 4 orificios de fijación, lo que facilita su uso en otros proyectos.



3. Parámetros técnicos

Shield	
Nombre	mShield
Referencia	M1E0002
USB	Placa base compatible
Micro USB	micro:bit
Pines	
Pines de E/S digitales	9 (P0, P1, P2, P8, P9, P13, P14, P15, P16)
Pines de lectura analógica	3 (P0, P1, P2)
Pines de escritura analógicos	9 (P0, P1, P2, P8, P9, P13, P14, P15, P16)
Pines PWM	4 (S1~, S2~, S3~, S4~), periodo: 2 ms.
Pines de servo	4 (S1~, S2~, S3~, S4~)
Comunicación	
UART	Sí
I2C	Sí
SPI	Sí
Receptor de infrarrojos	Sí (NEC)
Alimentación	
Tensión de entrada (nominal)	3–9 V
CC Corriente para el pin de 3,3 V	500 mA
Corriente de CC para el pin de 5 V	2000 mA
Motores	
Conectores (XH2,54 2P)	Motor 1, Motor 2
Tensión de salida	3–9 V
Corriente máxima de salida	1,2 A
Corriente de funcionamiento recomendada	1 A
LED	
Número	4 (20 %, 40 %, 60 %, 80 %)
Dimensiones	
Ancho	62 mm
Longitud	73 mm
Altura	14 mm
Peso	29,16 g

4. Especificaciones de la interfaz



Número	Descripción
1	Puerto micro USB, solo para alimentación externa.
2	Receptor de infrarrojos, utiliza el pin P12.
3	Ranura para micro:bit.
4	Indicadores LED, controlados por un chip I2C interno.
5	Toma de alimentación (DC005), admite una tensión de entrada de 3 a 9 V, se puede conectar a 3, 4, 5 y 6 pilas AA, o a 1 y 2 pilas de litio, y cuenta con función de protección contra conexión inversa.
6	Interruptor de la toma de corriente.
7, 8	Interfaces del motor (XH2.54 2P), se pueden conectar a motores de 3-9 V CC; la tensión del motor es igual a la tensión de la toma de corriente. Controladas por un chip I2C interno.
9, 10	Interruptor de selección de tensión de salida de los conectores V1 y V2.
11, 12	Puerto de expansión de E/S para Microbit.
13	Puerto de expansión interno de mShield, controlado por un chip I2C interno.
14	Interfaz I2C de alimentación de 3,3 V, P19 (SCL), P20 (SDA).
15	Interfaz I2C con alimentación de 5 V: P19 (SCL), P20 (SDA).

5. Conceptos básicos del editor de Python

El [editor de Python](#) es la forma perfecta de iniciarse en la programación con MicroPython en el BBC Micro:bit. El editor de Python es gratuito y funciona en un navegador en todas las plataformas.

Recomendamos utilizar los navegadores Google Chrome o Microsoft Edge. WebUSB es una función web de reciente desarrollo que permite acceder al Micro:bit directamente desde una página web. También permite enviar datos directamente desde el Micro:bit al editor de Python. Funciona con los navegadores Google Chrome y Microsoft Edge.

Versión de micro:bit compatible con WebUSB

Si no utilizas la última versión de Google Chrome o Microsoft Edge, asegúrate de que sea esta versión o una posterior:

Google Chrome (versión 79 y superiores) para Android, Chrome OS, Linux, macOS y Windows 10. Microsoft Edge (versión 79 y superiores) para Android, Chrome OS, Linux, macOS y Windows 10. Enlace para descargar la última versión del navegador Google Chrome:

<https://www.google.com/chrome/>

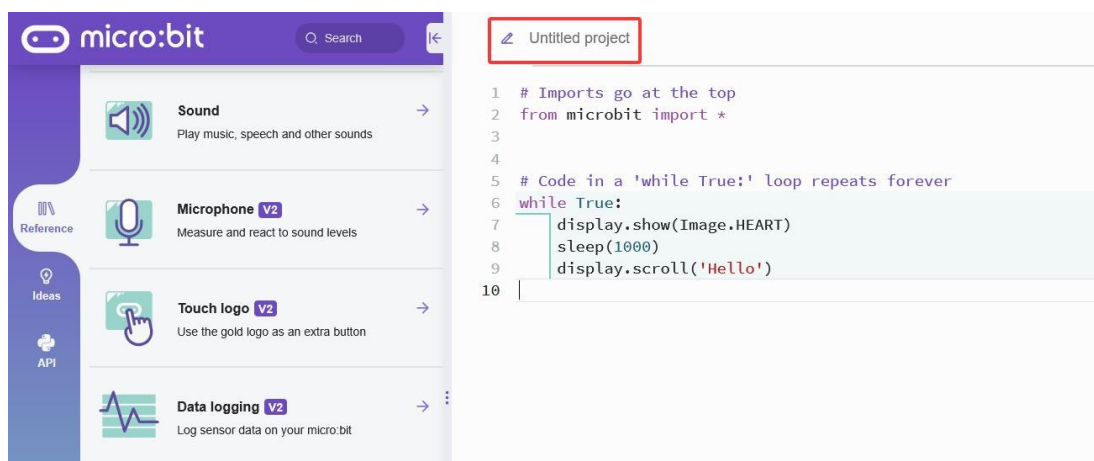
Enlace para descargar la última versión de Microsoft Edge:

<https://www.microsoft.com/en-us/edge/download>

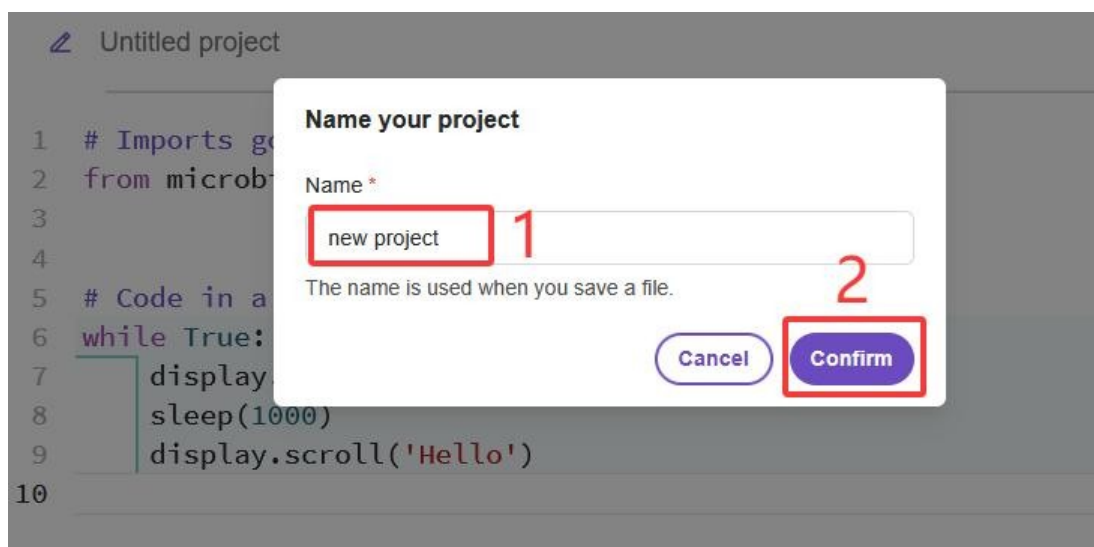
5.1 Crear un nuevo proyecto

Abre la página de programación en línea en el navegador Google Chrome del ordenador:

<https://python.microbit.org/v/3>, haz clic en el recuadro rojo:

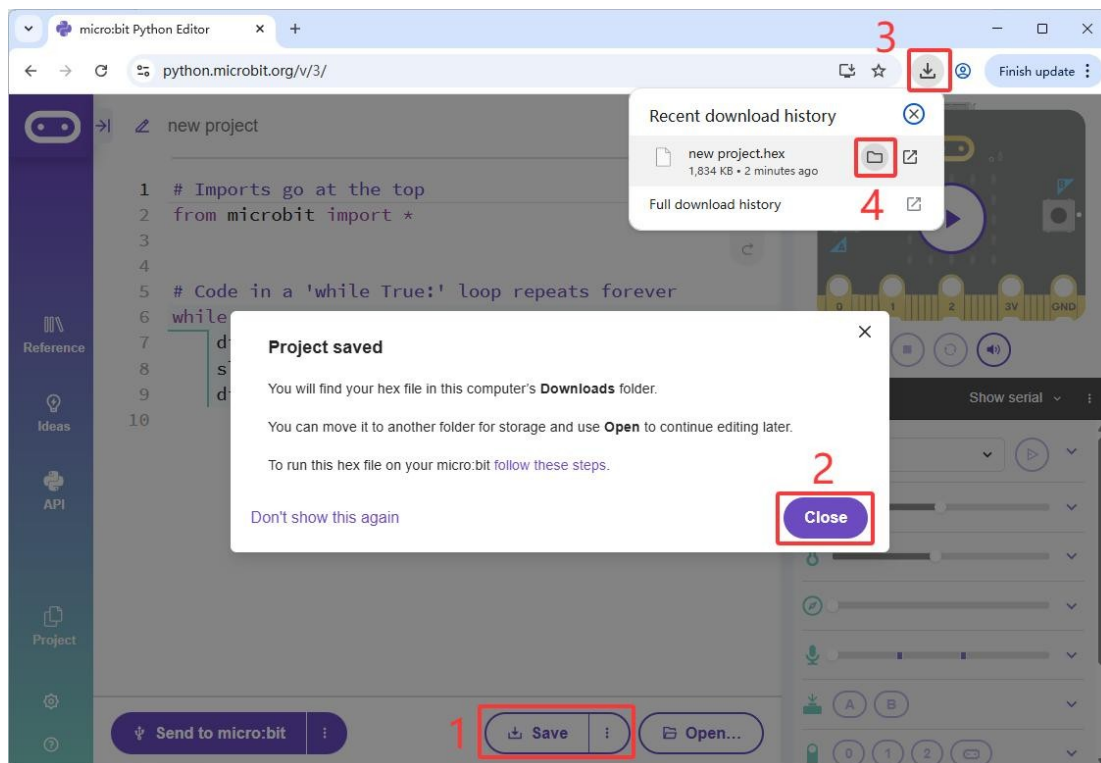


A continuación, introduce el nombre del proyecto y haz clic en «Confirmar»:



5.2 Guardar el proyecto localmente

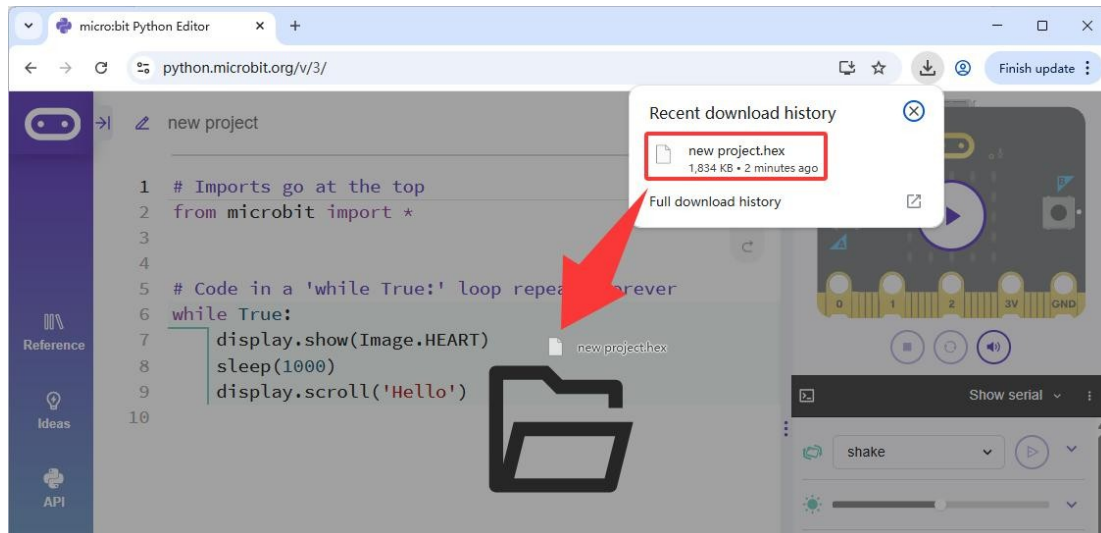
Haz clic en el botón «Guardar» del paso 1 y, a continuación, en el botón «Cerrar» del paso 2 para guardar el proyecto localmente. A continuación, sigue los pasos 3 y 4 para localizar el archivo «.hex» guardado, tal y como se muestra a continuación:



Nota: Si no quieres que vuelva a aparecer la ventana emergente, ¡haz clic en «No volver a mostrar esto» en el paso 2!

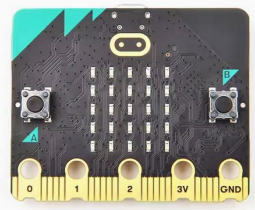
5.3 Importar el proyecto local

Arrastra directamente el archivo de proyecto «HEX» local al área de trabajo del editor de Python, tal y como se muestra a continuación:

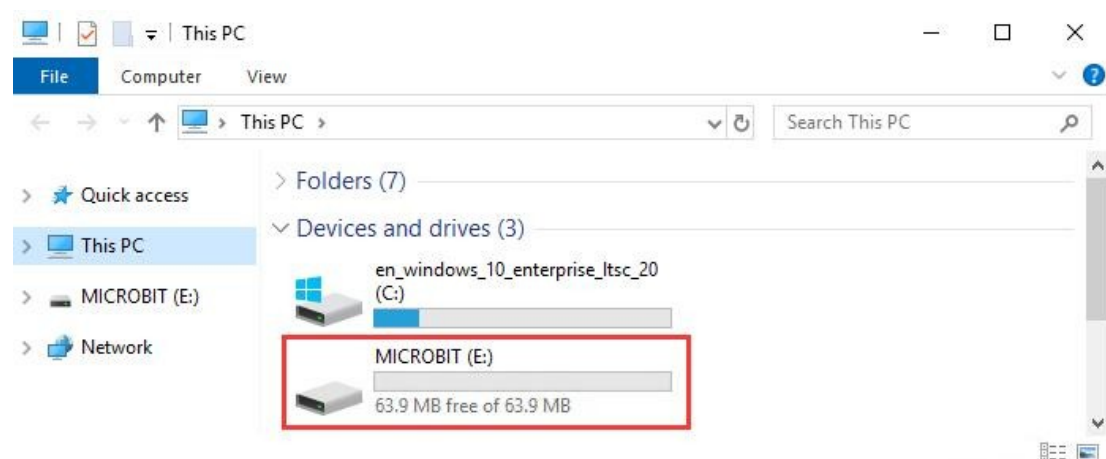
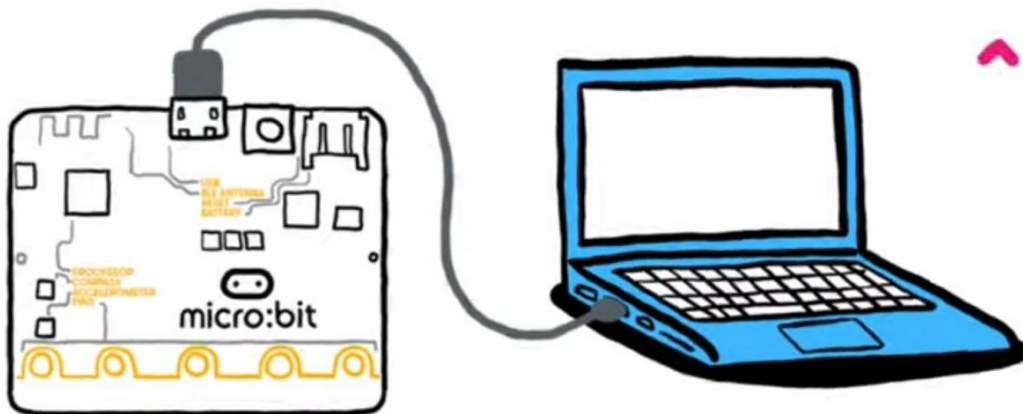


5.4 Subir proyecto

Herramientas:

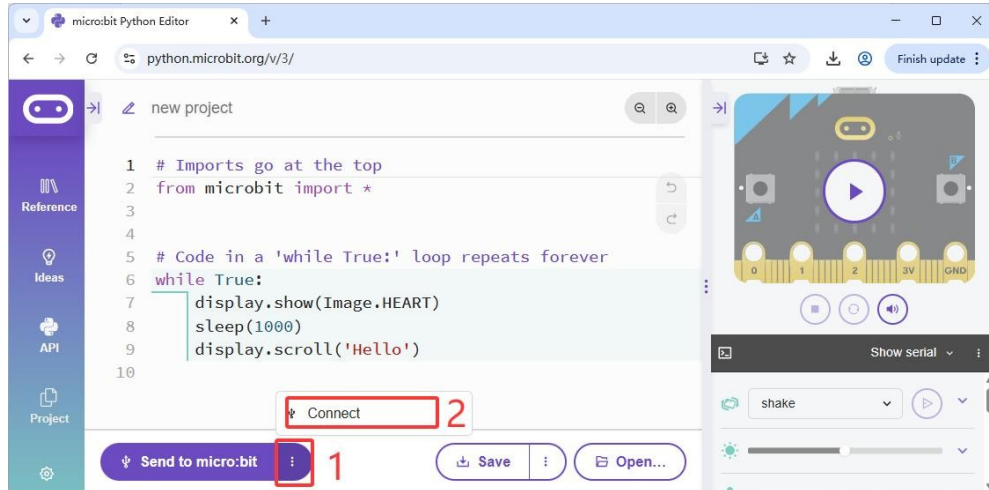
PC	micro:bit v2.x.x	Cable micro USB
		

Conecta la placa base del micro:bit al ordenador mediante un cable micro USB. Aparecerá una unidad de almacenamiento en el ordenador:

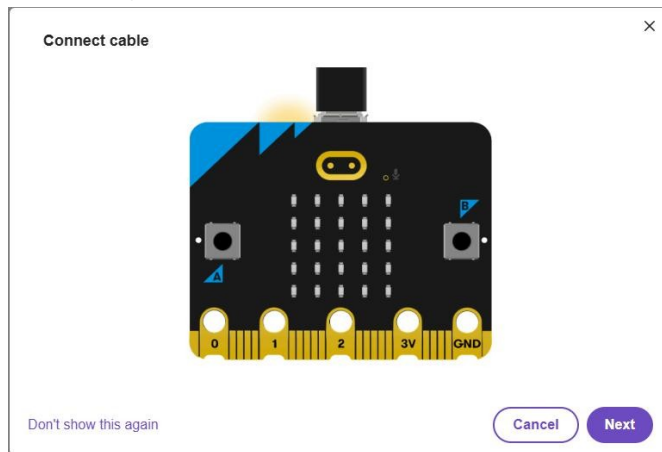


SIYEENOVE

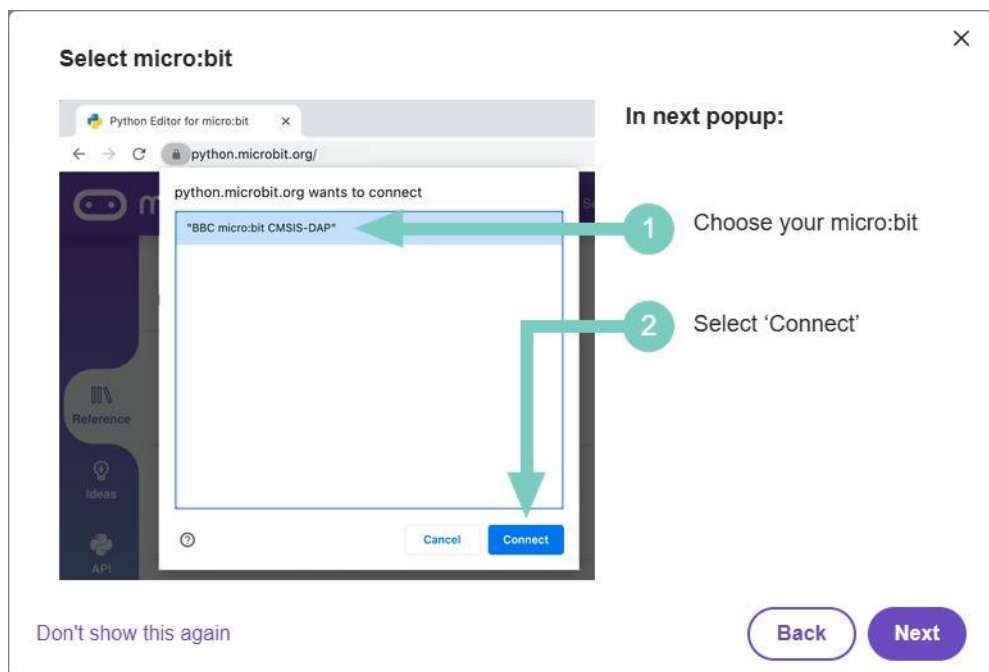
Haz clic en el botón «Conectar»:



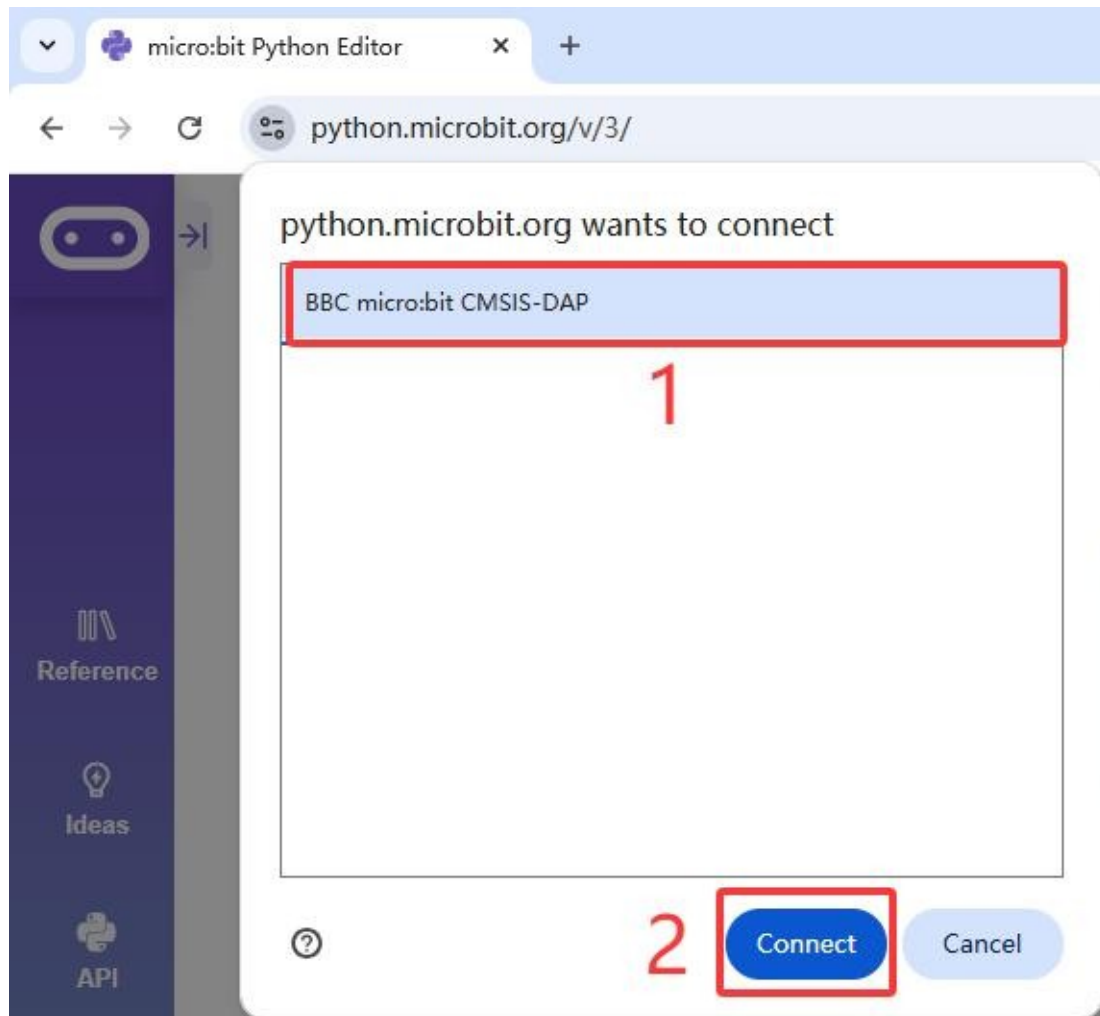
Haz clic en «Siguiente»:



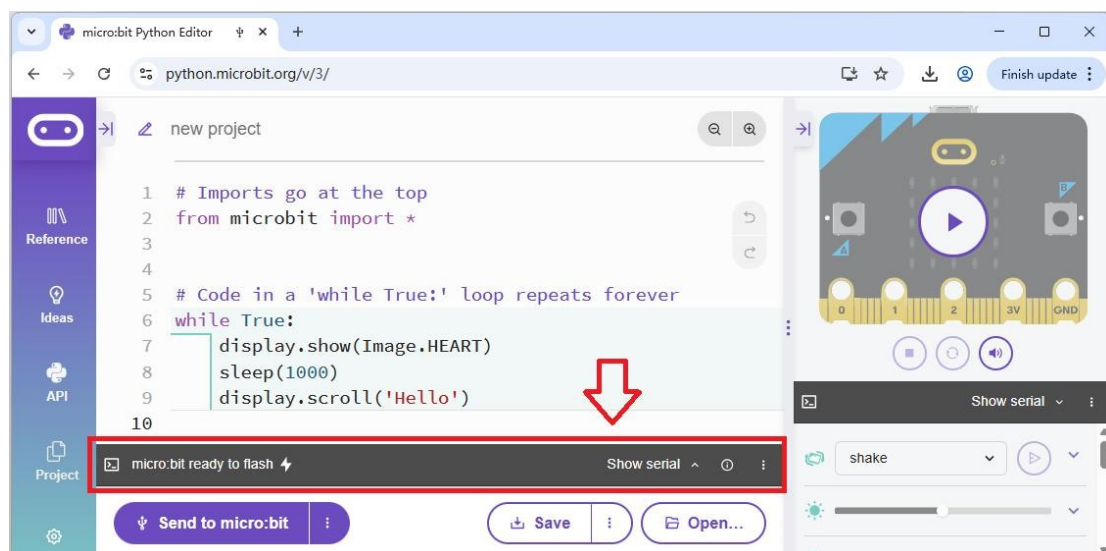
Haz clic en «Siguiente»:



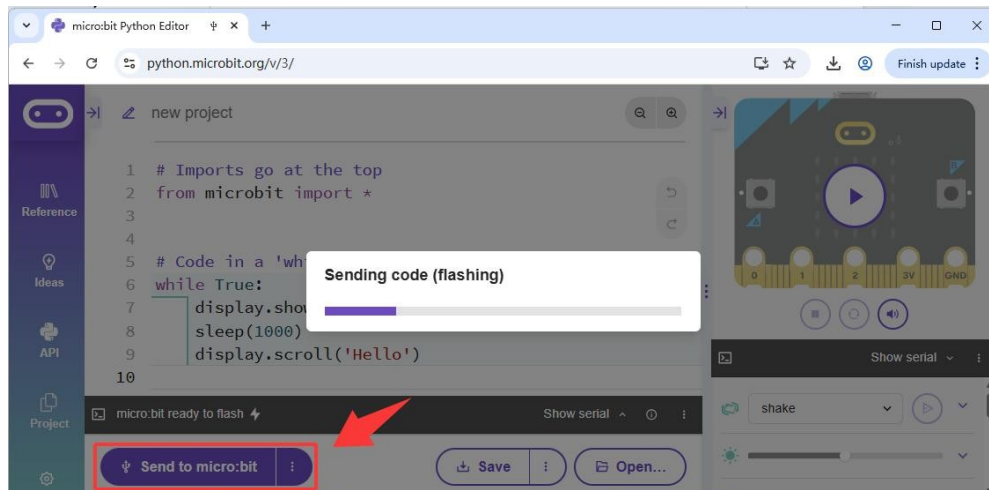
A continuación, sigue estos pasos:



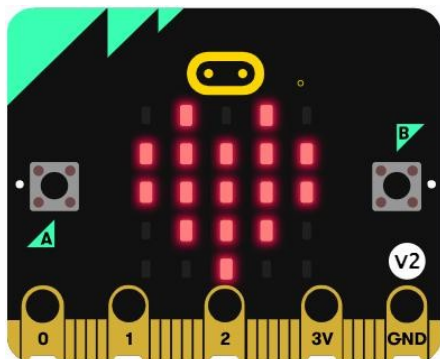
Cuando aparezca lo siguiente, significa que la conexión entre el micro:bit y el editor de Python se ha establecido correctamente:



Ahora haz clic en «Enviar a micro:bit» para cargar el código desde el editor de Python al micro:bit:

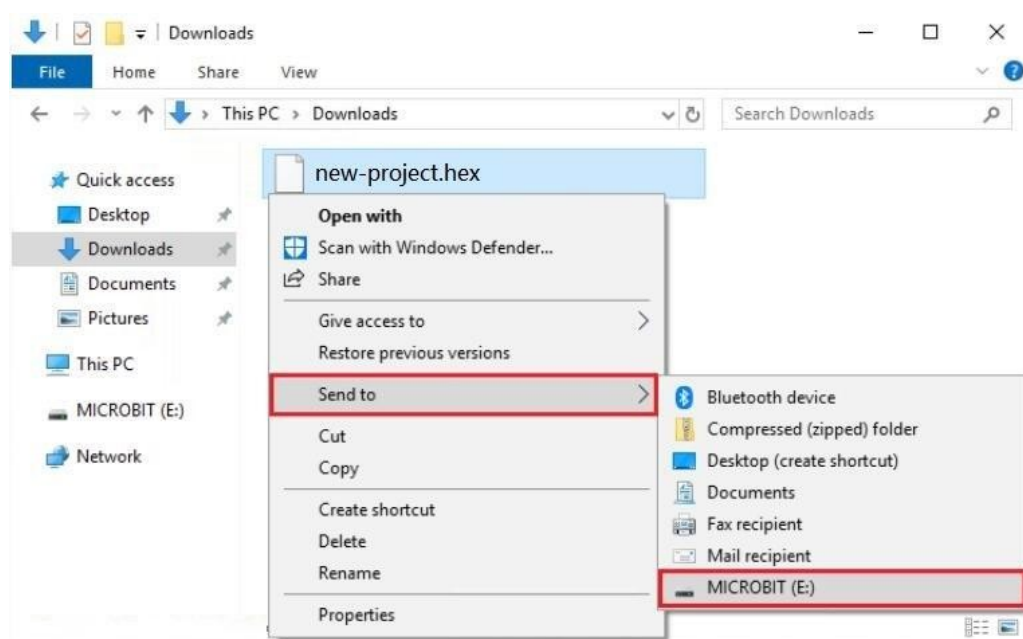


Una vez cargado el código, la pantalla de matriz de puntos de la placa base del micro:bit muestra una forma de corazón y, a continuación, aparece el mensaje «Hello»:



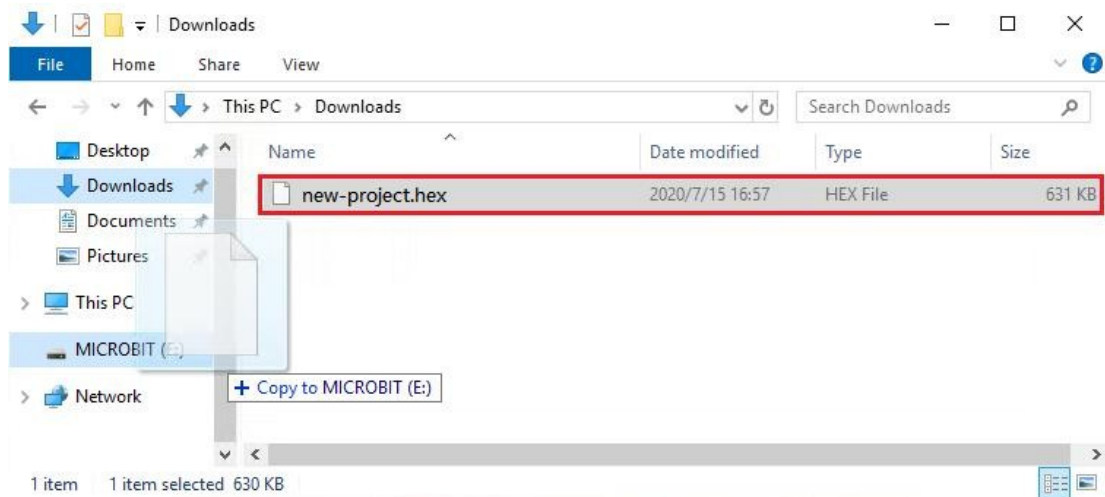
Otros métodos para cargar el código

Selecciona el archivo «HEX», haz clic con el botón derecho del ratón y envía el archivo «HEX» a la placa base del micro:bit:

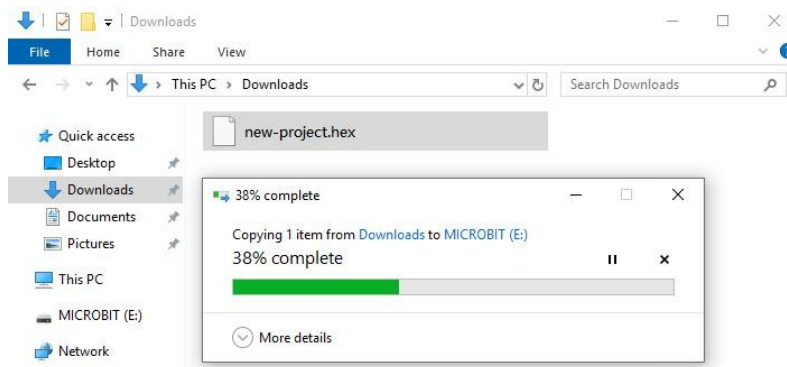


SIYEENOVE

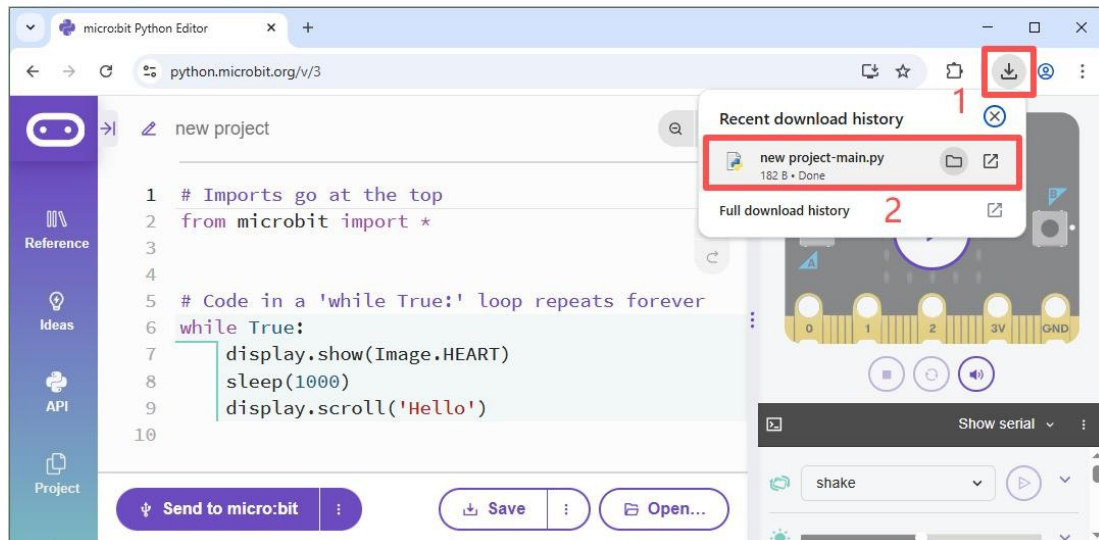
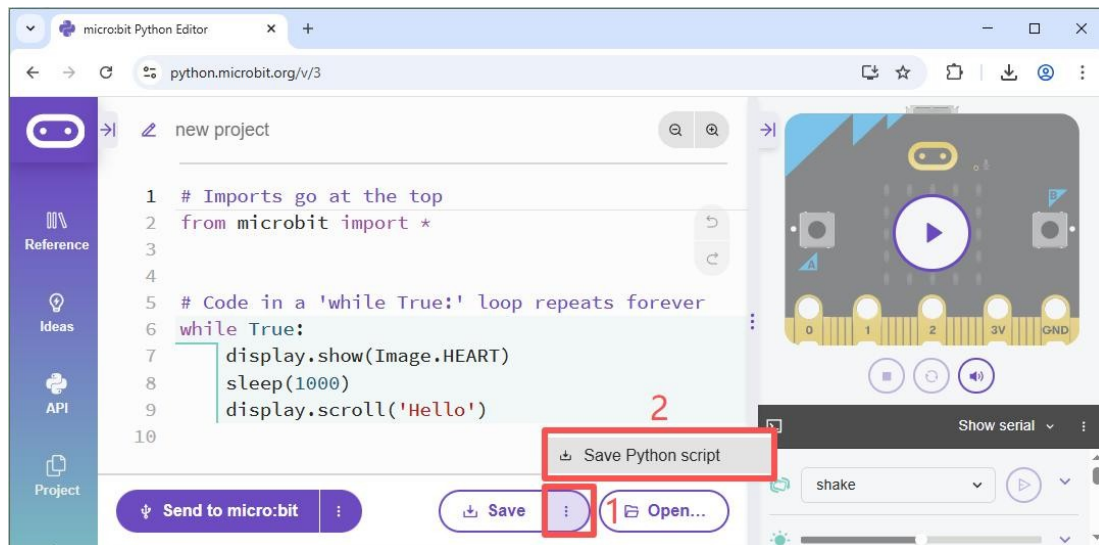
O bien, arrástralo y suéltalo directamente sobre la placa base del micro:bit:



La siguiente interfaz indica que el código se está cargando:



5.5 Guardar el proyecto como archivo Python



Nota: Los archivos Python «xx.py» no se pueden ejecutar directamente en el micro:bit; solo el archivo «xx.hex» del proyecto se puede ejecutar en el micro:bit. Los archivos «xx.py» se pueden abrir con cualquier editor de texto, mientras que los archivos «xx.hex» no.

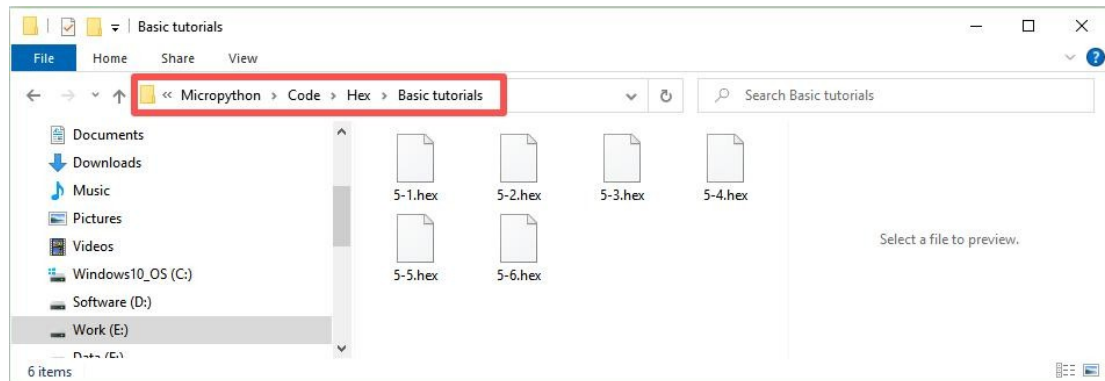
5.6 Aprendizaje de la sintaxis básica del editor de Python

La plataforma micro:bit ofrece documentación oficial de referencia sobre el uso del editor de Python. Guía del usuario: <https://microbit.org/get-started/user-guide/python-editor/>

Documentación de MicroPython: <https://microbit-micropython.readthedocs.io/en/v2-docs/>

6. Código de ejemplo

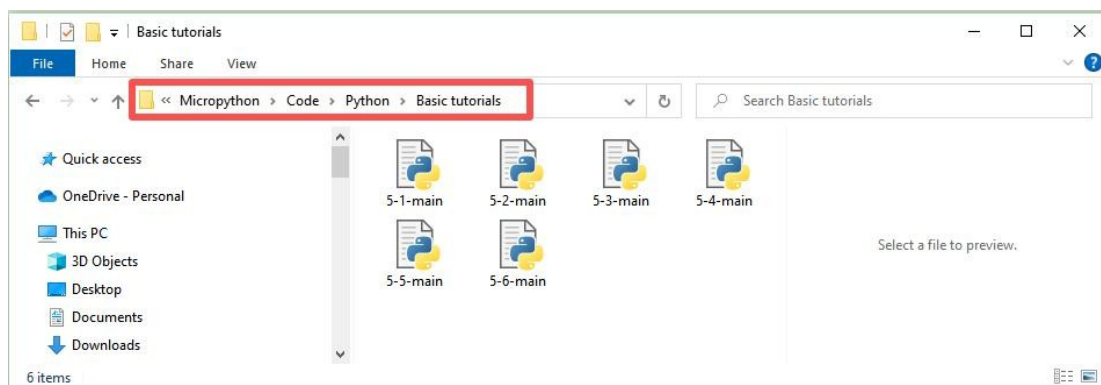
Los proyectos de ejemplo de los tutoriales básicos se guardan todos en la carpeta «Micropython -> Code -> Hex -> Basic tutorials»:



¡¡Aviso importante!!

Para los siguientes tutoriales, si deseas importar los proyectos de ejemplo locales que te proporcionamos, consulta la sección 5.3. Si deseas crear un nuevo proyecto, consulta la sección 5.1.

Los archivos de ejemplo en Python para los tutoriales básicos se guardan en la carpeta «MicroPython -> Código -> Python -> Tutoriales básicos»:

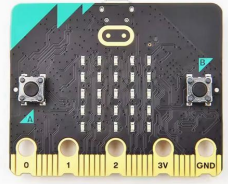


6.1 Versión del firmware

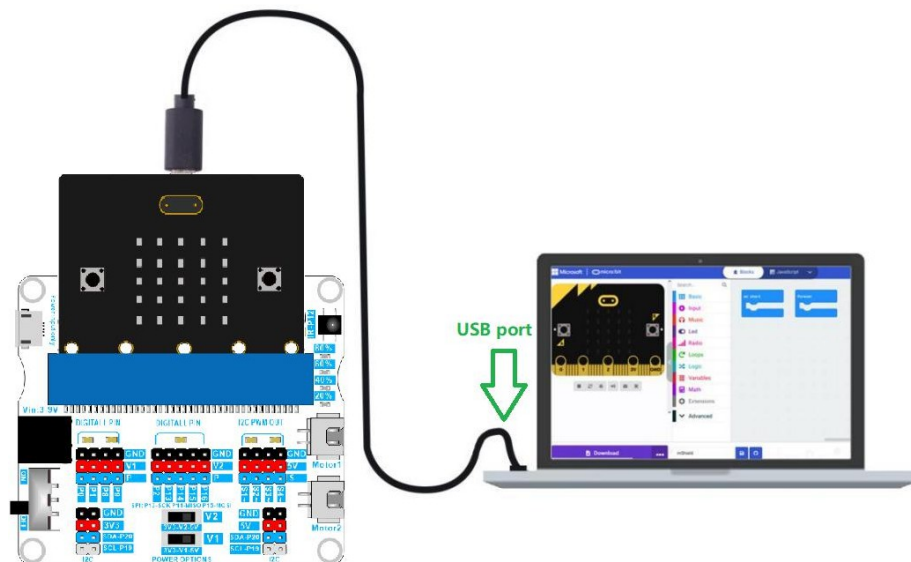
Objetivo

Leer la versión del firmware de la expansión mShield.

Herramientas:

PC	micro:bit V2.x.x	Cable USB
		

Cableado



Programación

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x29

# Firmware version register
ver = bytearray([0x00])

# Initialize the I2C communication interface
i2c.init()
```

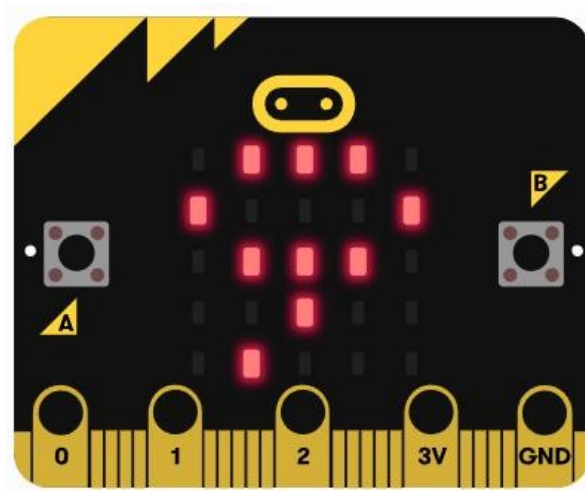


```
# Code in a 'while True:' loop repeats forever
while True:
    # Read the firmware version
    i2c.write(i2cAddr, ver, True)
    version = i2c.read(i2cAddr, 1)

    sleep(1000)          # Delay: 1000 milliseconds
    display.scroll(version[0]) # Scroll display of battery level
    sleep(3000)          # Delay: 3000 milliseconds
```

Conclusión

La placa de expansión mShield integra un microcontrolador de 8 bits para controlar motores, LED y los puertos S1-S4. Hemos precargado el firmware antes de salir de fábrica. Mediante el código anterior, puedes leer la versión del firmware y mostrar la versión del firmware en la matriz de LED del micro:bit.



Reflexiones

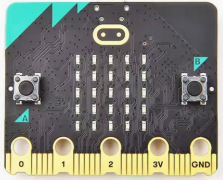
Problemas habituales

6.2 LED

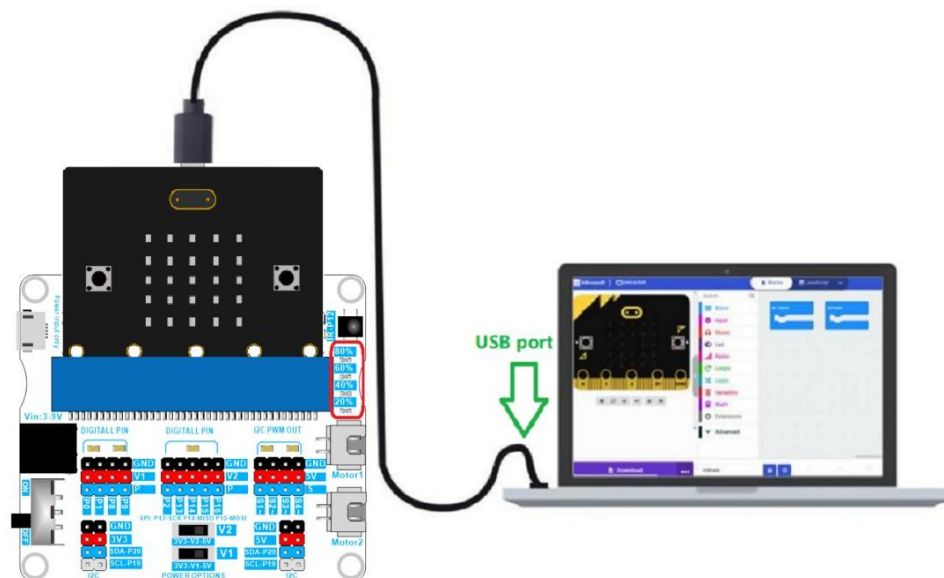
Objetivo

Encender los 4 LED de la placa de expansión.

Herramientas

PC	micro:bit V2.x.x	Cable USB
		

Cableado



Programación

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x29

# For LEDs
led1 = 11      #20%
led2 = 12      #40%
led3 = 13      #60%
led4 = 14      #80%
i2cBuf = bytearray([0x00, 0x00])
```

```

# Control the LED function
# led: led1 -- led4
# onOff: 0 = off, 1 = on
def setLed(led, onOff):
    if led == led1:
        i2cBuf[0] = led1
    elif led == led2:
        i2cBuf[0] = led2
    elif led == led3:
        i2cBuf[0] = led3
    elif led == led4:
        i2cBuf[0] = led4
    else:
        return
    i2cBuf[1] = onOff
    i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    setLed(led1, 1)    # led1 on
    sleep(1000)
    setLed(led1, 0)    # led1 off
    setLed(led2, 1)    # led2 on
    sleep(1000)
    setLed(led2, 0)    # led2 off
    setLed(led3, 1)    # led3 on
    sleep(1000)
    setLed(led3, 0)    # led3 off
    setLed(led4, 1)    # led4 on
    sleep(1000)
    setLed(led4, 0)    # led4 off

```

Conclusión

Los cuatro LED se encienden uno tras otro de forma cíclica.

Reflexión

Modifica el código anterior para que las luces cambien más rápido.

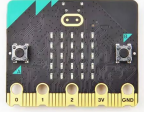
Problemas habituales

6.3 PWM

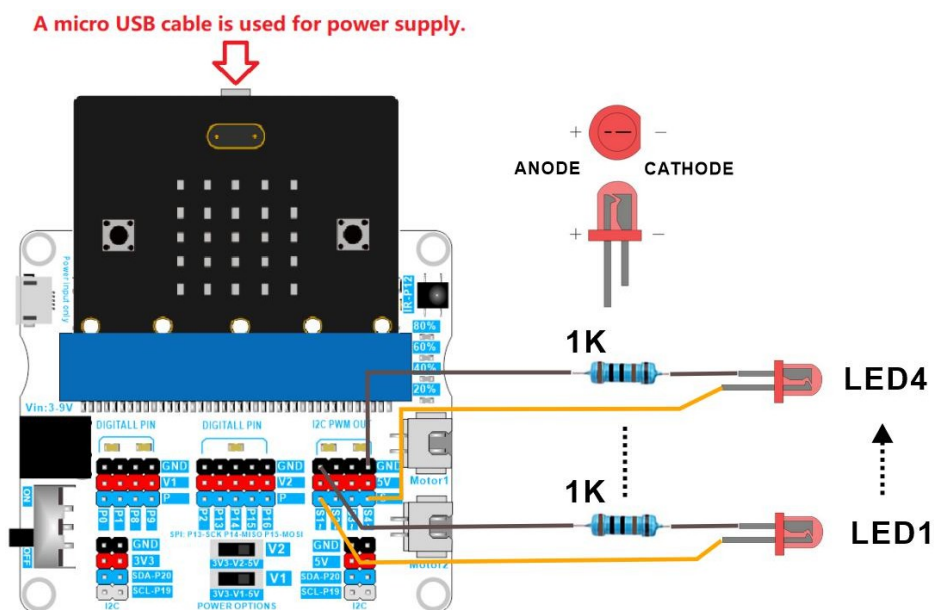
Objetivo

Utiliza los pines S1 a S4 para controlar los LED y cambiar su intensidad.

Herramientas

PC	micro:bit V2.x.x	Cable USB	LED
			
Resistencia de 1K	Cables Dupont hembra-hembra		
			

Cableado



Programación

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x29

# For Pins
s1 = 16      #S1 Pin
```

```

s2 = 17      #S2 Pin
s3 = 18      #S3 Pin
s4 = 19      #S4 Pin
i2cBuf = bytearray([0x00, 0x00])

# S1--S4 pins mode
s1ToS4 = 15
pwmMode = 1
servoMode = 2

# Control S1--S4 pins output PWM signal function
# pin: s1 -- s4
# pwm: 0 -- 200, map:0 -- 2000us, period:2000us
def setPinPwm(pin, pwm):
    if pin == s1:
        i2cBuf[0] = s1
    elif pin == s2:
        i2cBuf[0] = s2
    elif pin == s3:
        i2cBuf[0] = s3
    elif pin == s4:
        i2cBuf[0] = s4
    else:
        return
    i2cBuf[1] = pwm
    i2c.write(i2cAddr, i2cBuf)

# Set the S1--S4 pin mode
# mode: 1 = PWM mode, 2 = servo mode
def setPinMode(mode):
    if mode == servoMode:
        i2cBuf[1] = servoMode
    elif mode == pwmMode:
        i2cBuf[1] = pwmMode
    else:
        return
    i2cBuf[0] = s1ToS4
    i2c.write(i2cAddr, i2cBuf)

# Set the pins S1--S4 to PWM mode
setPinMode(pwmMode)

# Code in a 'while True:' loop repeats forever
while True:

```

```
for pwm in range(200):
    setPinPwm(s1, pwm)
    setPinPwm(s2, pwm)
    setPinPwm(s3, pwm)
    setPinPwm(s4, pwm)
    sleep(10)
```

Conclusión

El brillo de LED1, LED2, LED3 y LED4 cambia lentamente de oscuro a brillante.

Reflexión

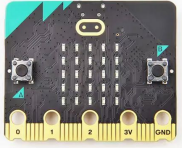
Problemas habituales

6.4 Nivel de batería

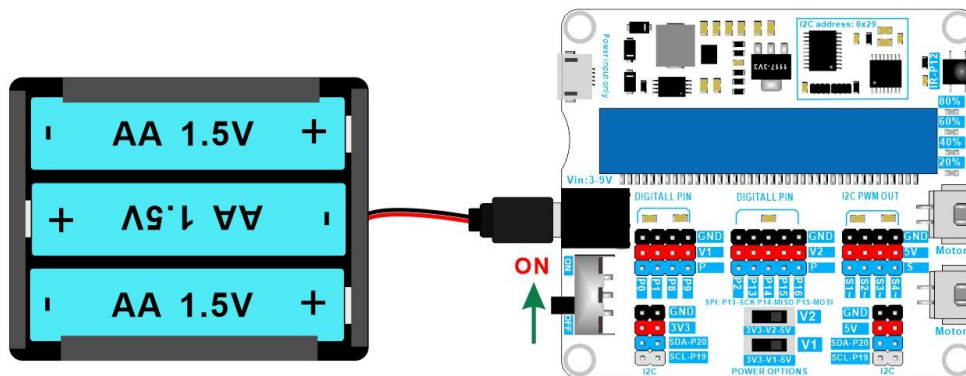
Objetivo

Leer el nivel de batería del mShield.

Herramientas

PC	micro:bit V2.x.x	Cable USB	Alimentación con 3 pilas AA
			

Cableado



Programación

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x29

# Type of battery
aaBattery3 = bytearray([0x01])    # 3 AA batteries
aaBattery4 = bytearray([0x02])    # 4 AA batteries
aaBattery5 = bytearray([0x03])    # 5 AA batteries
aaBattery6 = bytearray([0x04])    # 6 AA batteries
lithiumBattery1 = bytearray([0x05]) # 1 lithium battery
lithiumBattery2 = bytearray([0x06]) # 2 lithium batteries

# Initialize the I2C communication interface
i2c.init()

# Code in a 'while True:' loop repeats forever
while True:
    # Read the level of 3 AA batteries
    i2c.write(i2cAddr, aaBattery3, True)
    batLevel = i2c.read(i2cAddr, 1)

    sleep(1000)                # Delay: 1000 milliseconds
    display.scroll(batLevel[0]) # Scroll display of battery level
    sleep(3000)                # Delay: 3000 milliseconds
```

Conclusión

Cuando el mShield se conecta a tres pilas AA en serie, el micro:bit puede leer el nivel de batería (0-100 %) y mostrar este valor en la matriz LED del micro:bit.

Reflexión

Combina esta información con la sección 6.2 para escribir un programa de indicador de nivel de batería, en el que los 4 LED representen los niveles de batería del 20 %, 40 %, 60 % y 80 %, respectivamente.

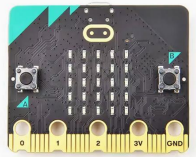
Problemas habituales

6.5 Motor

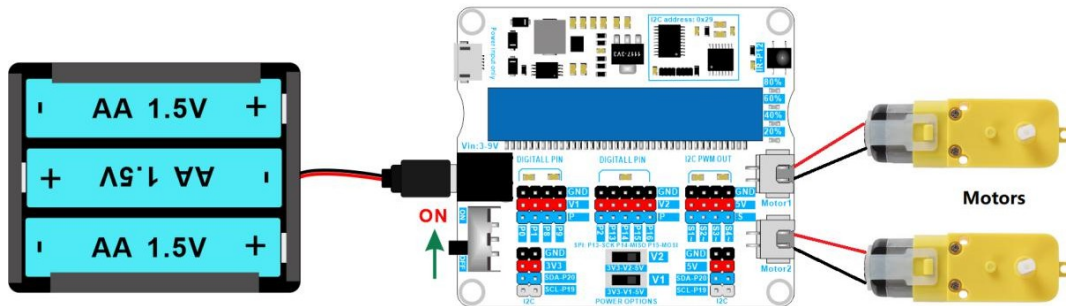
Objetivo

Utiliza las dos interfaces de motor de la placa de expansión para controlar dos motores.

Herramientas

PC	micro:bit V2.x.x	Cable USB	Motor
			
Alimentación con 3 pilas AA			
			

Cableado



Programación

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x29

# For motor
motor1 = 1
motor2 = 2
CW = 1 # clockwise
CCW = 2 # counterclockwise
i2cBuf = bytearray([0x00, 0x00])

# Set the motor speed function
# motor: 1 = motor1, 2 = motor2
# direction: 1 = CW, 2 = CCW
# speed: 0--100
def setMotorSpeed(motor, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if motor == motor1:
        i2cBuf[0] = 0x09 # motor1 register
        if direction == CW:
            i2cBuf[1] = speed
        elif direction == CCW:
            # speed value, 101 is the default required data.
```



```

if motor == motor2:
    i2cBuf[0] = 0x0a # motor2 register
    if direction == CW:
        i2cBuf[1] = speed
    elif direction == CCW:
        # speed value, 101 is the default required data.
        i2cBuf[1] = speed + 101
    i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    # CW
    setMotorSpeed(motor1, CW, 100)
    setMotorSpeed(motor2, CW, 100)
    sleep(1000)
    # stop
    setMotorSpeed(motor1, CW, 0)
    setMotorSpeed(motor2, CW, 0)
    sleep(1000)
    # CCW
    setMotorSpeed(motor1, CCW, 100)
    setMotorSpeed(motor2, CCW, 100)
    sleep(1000)
    # stop
    setMotorSpeed(motor1, CCW, 0)
    setMotorSpeed(motor2, CCW, 0)
    sleep(1000)

```

Conclusión

Cuando se conectan motores externos a las interfaces Motor1 y Motor2, el motor 1 y el motor 2 giran primero en sentido horario durante 1 segundo y, a continuación, se detienen durante 1 segundo; después, giran en sentido antihorario durante 1 segundo y se detienen durante 1 segundo, repitiendo este ciclo de forma continua.

Reflexión

Problemas habituales

Si el valor de compensación no viene configurado de fábrica, es posible que el motor no gire. ¡Basta con restablecer el valor de compensación para resolver este problema!

Si las velocidades de los dos motores conectados a las interfaces de motor de la placa de expansión son diferentes debido a diferencias de hardware entre los motores, aunque los valores de velocidad sean los mismos, podemos utilizar la siguiente instrucción para resolver esta discrepancia. Los parámetros de velocidad compensados se pueden guardar de forma permanente en el mShield.

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x29
i2cBuf = bytearray([0x00, 0x00])

# For wheel
leftwheel = 0
rightwheel = 1

# Calibrate the wheel speed function
# leftWheelOffset: 0--10
# rightWheelOffset: 0--10
def wheelsAdjustment(leftWheelOffset, rightWheelOffset):
    # The limit variable is between 0 and 10.
    leftWheelOffset = max(0, min(10, leftWheelOffset))
    rightWheelOffset = max(0, min(10, rightWheelOffset))

    i2cBuf[0] = 0x07 # Left wheel offset register
    i2cBuf[1] = leftWheelOffset
    i2c.write(i2cAddr, i2cBuf)
    i2cBuf[0] = 0x08 # Right wheel offset register
    i2cBuf[1] = rightWheelOffset
    i2c.write(i2cAddr, i2cBuf)

# The compensation values of the left and right motors are set to 0
wheelsAdjustment(0, 0)
```

```
while True:
    None
```

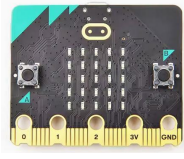
La compensación de la velocidad del motor se consigue reduciendo la velocidad de uno de los motores para igualar las velocidades de ambos. El rango de valores de compensación es de 0 a 10. Esta instrucción solo debe ejecutarse una vez durante la ejecución del programa, y los parámetros de velocidad compensada pueden guardarse de forma permanente en el mShield. Las implementaciones posteriores del código no necesitan incluir esta instrucción, a menos que sea necesario realizar un ajuste adicional de la velocidad de compensación del motor.

6.6 Servo de 180 grados

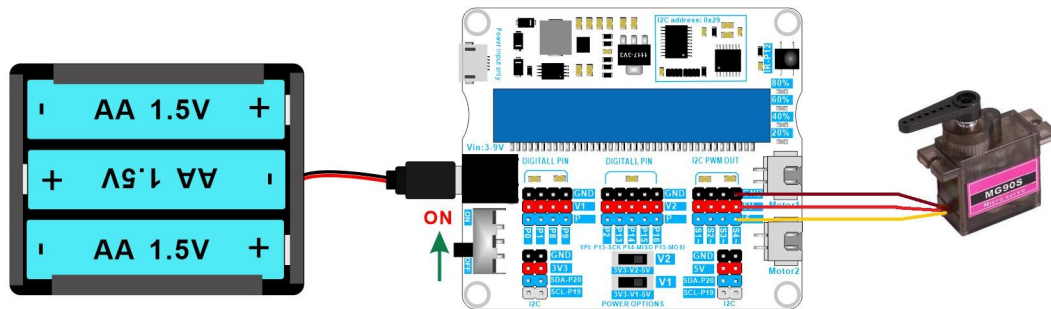
Objetivo

Aprender a controlar un servo de 180 grados.

Herramientas

PC	micro:bit V2.x.x	Cable USB	Servo de 180 grados
			
Alimentación con 3 pilas AA			
			

Cableado



Programación

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x29
i2cBuf = bytearray([0x00, 0x00])

# S1--S4 pins mode
s1ToS4 = 15
pwmMode = 1
servoMode = 2

# Servo type
servo90 = 0
servo180 = 1
servo270 = 2
pinS1 = 20    # S1 pin
pinS2 = 21    # S2 pin
pinS3 = 22    # S3 pin
pinS4 = 23    # S4 pin

# Set the S1--S4 pin mode
# mode: 1 = PWM mode, 2 = servo mode
def setPinMode(mode):
    if mode == servoMode:
        i2cBuf[1] = servoMode
    elif mode == pwmMode:
        i2cBuf[1] = pwmMode
    else:
        return
```

```

i2cBuf[0] = s1ToS4
i2c.write(i2cAddr, i2cBuf)

# Set the Angle function of 90, 180 and 270 servo.
# index: 20 = S1 pin, 21 = S2 pin, 22 = S3 pin, 23 = S4 pin
# servoType: 0 = 90 servo, 1 = 180 servo, 2 = 270 servo
# angle: 90 servo -> 0-90, 180 servo -> 0-180, 270 servo -> 0-270
def setServo(index, servoType, angle):
    angleMap = 0
    if servoType == servo90:
        # Map 0-90 to 50-250 # 50 -- 250 map 500us -- 2500us, period:20000us
        angleMap = scale(angle, from_=(0, 90), to=(50, 250))
    if servoType == servo180:
        # Map 0-90 to 50-250
        angleMap = scale(angle, from_=(0, 180), to=(50, 250))
    if servoType == servo270:
        # Map 0-90 to 50-250
        angleMap = scale(angle, from_=(0, 270), to=(50, 250))

    if index == pinS1:
        i2cBuf[0] = pinS1 # S1 pin
    if index == pinS2:
        i2cBuf[0] = pinS2 # S2 pin
    if index == pinS3:
        i2cBuf[0] = pinS3 # S3 pin
    if index == pinS4:
        i2cBuf[0] = pinS4 # S4 pin

    i2cBuf[1] = angleMap
    i2c.write(i2cAddr, i2cBuf)

# Set the pins S1--S4 to servo mode
setPinMode(servoMode)

# Code in a 'while True:' loop repeats forever
while True:
    # The 180 degree servo of the S1 pin turns to the 0 degree position
    setServo(pinS1, servo180, 0)
    # The 180 degree servo of the S2 pin turns to the 0 degree position
    setServo(pinS2, servo180, 0)
    # The 180 degree servo of the S3 pin turns to the 0 degree position
    setServo(pinS3, servo180, 0)
    # The 180 degree servo of the S4 pin turns to the 0 degree position
    setServo(pinS4, servo180, 0)

```

SIYEENOVE

```
sleep(1000)
```

```
# The 180 degree servo of the S1 pin turns to the 180 degree position
setServo(pinS1, servo180, 180)
# The 180 degree servo of the S2 pin turns to the 180 degree position
setServo(pinS2, servo180, 180)
# The 180 degree servo of the S3 pin turns to the 180 degree position
setServo(pinS3, servo180, 180)
# The 180 degree servo of the S4 pin turns to the 180 degree position
setServo(pinS4, servo180, 180)
sleep(1000)
```

Conclusión

Los servos oscilan entre 0 y 180 grados.

Reflexión

¿Cómo controlar servos de 90 y 270 grados?

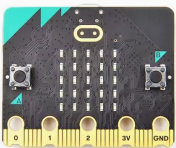
Problemas habituales

6.7 Servo de 360 grados

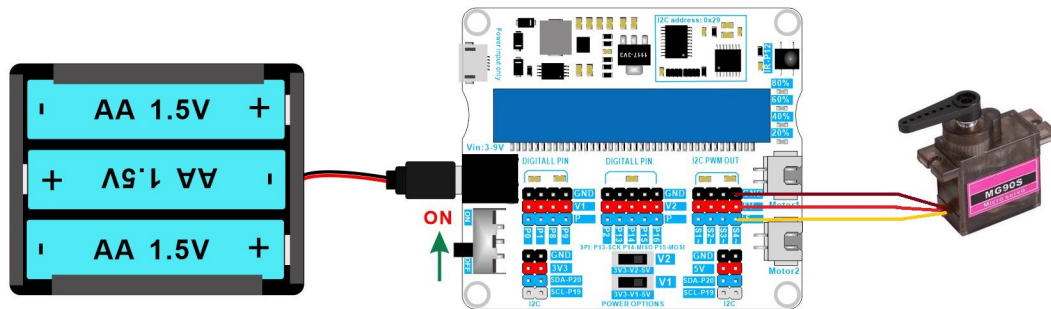
Objetivo

Aprender a controlar un servo de 360 grados.

Herramientas

PC	micro:bit V2.x.x	Cable USB	Servo de 360 grados
			
Alimentación con 3 pilas AA			
			

Cableado



Programación

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x29
i2cBuf = bytearray([0x00, 0x00])

# S1--S4 pins mode
s1ToS4 = 15
pwmMode = 1
servoMode = 2

# Servo type
servo90 = 0
servo180 = 1
servo270 = 2
pinS1 = 20    # S1 pin
pinS2 = 21    # S2 pin
pinS3 = 22    # S3 pin
pinS4 = 23    # S4 pin

# Set the S1--S4 pin mode
# mode: 1 = PWM mode, 2 = servo mode
def setPinMode(mode):
    if mode == servoMode:
        i2cBuf[1] = servoMode
    elif mode == pwmMode:
        i2cBuf[1] = pwmMode
    else:
        return
```

```

# Set the Angle function of 90, 180 and 270 servo.
# index: 20 = S1 pin, 21 = S2 pin, 22 = S3 pin, 23 = S4 pin
# servoType: 0 = 90 servo, 1 = 180 servo, 2 = 270 servo
# angle: 90 servo -> 0-90, 180 servo -> 0-180, 270 servo -> 0-270
def setServo(index, servoType, angle):
    angleMap = 0
    if servoType == servo90:
        # Map 0-90 to 50-200
        angleMap = scale(angle, from_=(0, 90), to=(50, 200))
    if servoType == servo180:
        # Map 0-90 to 50-200
        angleMap = scale(angle, from_=(0, 180), to=(50, 200))
    if servoType == servo270:
        # Map 0-90 to 50-200
        angleMap = scale(angle, from_=(0, 270), to=(50, 200))

    if index == pinS1:
        i2cBuf[0] = pinS1 # S1 pin
    if index == pinS2:
        i2cBuf[0] = pinS2 # S2 pin
    if index == pinS3:
        i2cBuf[0] = pinS3 # S3 pin
    if index == pinS4:
        i2cBuf[0] = pinS4 # S4 pin

    i2cBuf[1] = angleMap
    i2c.write(i2cAddr, i2cBuf)

# Set the speed of 360 servo
# index: 20 = S1 pin, 21 = S2 pin, 22 = S3 pin, 23 = S4 pin
# speed: -100 to +100
def setServo360(index, speed):
    # Map -100 - 100 to 0 - 180
    angle = scale(speed, from_=(-100, 100), to=(0, 180))
    setServo(index, servo180, angle)

# Set the pins S1--S4 to servo mode
setPinMode(servoMode)

# Code in a 'while True:' loop repeats forever
while True:

```



```
# The 360-degree servo of the S1--S4 pins is forward
# at the maximum speed
setServo360(pinS1, 100)
setServo360(pinS2, 100)
setServo360(pinS3, 100)
setServo360(pinS4, 100)
sleep(1000)

# The 360-degree servo of the S1--S4 pins is backward
# at the maximum speed
setServo360(pinS1, -100)
setServo360(pinS2, -100)
setServo360(pinS3, -100)
setServo360(pinS4, -100)
sleep(1000)
```

Conclusión

Cuando se conectan servos de 360 grados externamente a los pines S1, S2, S3 y S4, los servos realizan ciclos hacia delante y hacia atrás a velocidad máxima con intervalos de 1 segundo.

Reflexión

¿Cómo hacer que el servo de 360 grados arranque suavemente?

Problemas habituales

7. Preguntas frecuentes

7.1 No se puede cargar el código en el micro:bit

👉 ¿Estás utilizando un cable USB con función de comunicación de datos?

👉 ¿Está el cable USB en buen estado?

7.2 La letra de la unidad de micro:bit se muestra incorrectamente

👉 La letra de la unidad aparece como: MAINTENANCE (normalmente MICROBIT). Esto ocurre porque se pulsó accidentalmente el botón de reinicio del micro:bit al encenderlo, lo que provocó que entrara en modo de actualización del firmware. Vuelve a actualizar el firmware para que vuelva a la normalidad. Puedes consultar el siguiente enlace para ver cómo hacerlo:

<https://microbit.org/get-started/user-guide/firmware/>

7.3 El motor sigue girando tras volver a cargar el código

👉 Esto se debe a que no se volvió a llamar a la función para detener el motor de la sección 6.5:

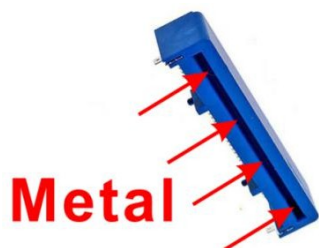
```
setMotorSpeed(wheel, direction, 0)
```

7.4 El motor no funciona

🔊 Asegúrate de que el micro:bit esté completamente insertado en la ranura del mShield.

🔊 Asegúrate de que el conector lateral del micro:bit esté limpio.

🔊 Asegúrate de que la ranura del mShield esté limpia.



👉 Consulta los problemas habituales en la sección 6.5 y restablece el valor de compensación de la rueda a 0.

7.5 Otros fallos

- ☛ ¿Ha comprobado que el montaje sea correcto?
- ☛ Comprueba si la carga de las pilas es suficiente.
- ☛ Compruebe si las pilas utilizadas cumplen con las especificaciones.

8. Contáctanos

Si no encuentra la solución en los puntos anteriores, póngase en contacto con nuestro equipo de asistencia.

Para que podamos atenderle rápidamente, tenga a mano la siguiente información:

Su número de pedido.

Modelo del producto (por ejemplo, placa de expansión mShield M1E0002) y software (por ejemplo, MakeCode o MicroPython).

Una descripción detallada del problema o la pregunta. Los pasos que ya ha intentado seguir.

Cualquier captura de pantalla de mensajes de error, fotos o fragmentos de código relevantes.

¿Alguna otra consulta?

Valoramos mucho tus comentarios y siempre buscamos mejorar. No dudes en ponerte en contacto con nosotros:

Errores en los tutoriales y comentarios: Ayúdanos a mejorar nuestra documentación.

Ideas y sugerencias sobre productos: nos encantaría conocer tus ideas. Asociaciones y colaboraciones: ¿te interesa colaborar? Hablemos. Descuentos y promociones: consulta los precios para el sector educativo o por volumen. Cualquier otra cosa: para todas las demás preguntas no técnicas.

Canales de asistencia



support@siyeenove.com



<https://siyeenove.com>